

LAPORAN TUGAS 6
DEADLOCK



Dosen Pengampu Mata Kuliah:
Triawan Adi Cahyanto, M.Kom

Disusun Oleh:
Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH
JEMBER 2025

1. Tugas 1 Analisis Deadlock

Jalankan program *deadlock_simple.c* dan jelaskan:

1. Mengapa program mengalami deadlock?

Program mengalami deadlock karena kedua thread saling menunggu resource yang dipegang oleh thread lain.

- Thread 1 memegang lock1, lalu ingin lock2.
- Thread 2 memegang lock2, lalu ingin lock1.

Tidak ada thread yang bisa melanjutkan, sehingga program macet (hang) selamanya.

2. Kondisi deadlock apa saja yang terpenuhi?

Semua 4 kondisi Coffman terpenuhi:

1. Mutual Exclusion

Lock adalah resource eksklusif, hanya satu thread dapat memegang mutex pada satu waktu.

2. Hold and Wait

Thread 1 memegang lock1 dan menunggu lock2 — Thread 2 memegang lock2 dan menunggu lock1.

3. No Preemption

Mutex tidak bisa direbut paksa; thread hanya bisa melepas sendiri.

4. Circular Wait

T1 → menunggu T2

T2 → menunggu T1

Ini membentuk lingkaran tunggu (circular wait).

3. Gambar diagram resource allocation graph-nya

Thread 1 ----(memegang)----> Lock1

Thread 2 ----(memegang)----> Lock2

Thread 1 ----(menunggu)----> Lock2

Thread 2 ----(menunggu)----> Lock1

Graf menunjukkan siklus → **deadlock terjadi.**

2. Tugas 2 Implementasi Pencegahan

Modifikasi program *deadlock_simple.c* menggunakan teknik:

1. Lock ordering

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

// Semua thread WAJIB mengambil lock dengan urutan: lock1 → lock2
void acquire_ordered_locks() {
    pthread_mutex_lock(&lock1);
    printf("Berhasil mengambil Lock 1\n");

    pthread_mutex_lock(&lock2);
    printf("Berhasil mengambil Lock 2\n");
}

void release_ordered_locks() {
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
}

void* thread_func(void* arg) {
    printf("Thread mulai bekerja\n");

    acquire_ordered_locks();

    printf("Thread sedang menggunakan resource...\n");
    sleep(1);

    release_ordered_locks();

    printf("Thread selesai\n");
    return NULL;
}
```

```

int main() {
    pthread_t t1, t2;

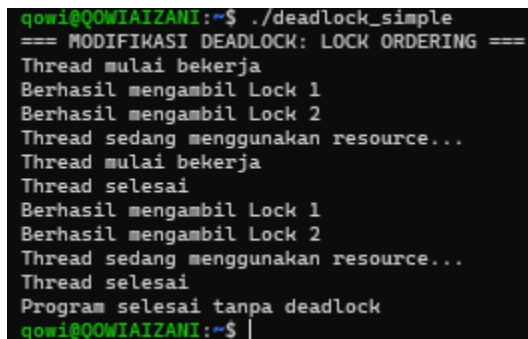
    printf("=== MODIFIKASI DEADLOCK: LOCK ORDERING ===\n");

    pthread_create(&t1, NULL, thread_func, NULL);
    pthread_create(&t2, NULL, thread_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai tanpa deadlock\n");
    return 0;
}

```



```

qowi@QOWIAIZANI:~$ ./deadlock_simple
=== MODIFIKASI DEADLOCK: LOCK ORDERING ===
Thread mulai bekerja
Berhasil mengambil Lock 1
Berhasil mengambil Lock 2
Thread sedang menggunakan resource...
Thread mulai bekerja
Thread selesai
Berhasil mengambil Lock 1
Berhasil mengambil Lock 2
Thread sedang menggunakan resource...
Thread selesai
Program selesai tanpa deadlock
qowi@QOWIAIZANI:~$

```

2. Try-lock dengan retry

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void* thread_func1(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock1);
        printf("Thread 1: mengambil Lock 1\n");

        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: berhasil mengambil kedua lock!\n");

```

```

        sleep(1);

        pthread_mutex_unlock(&lock2);
        pthread_mutex_unlock(&lock1);
        break;
    } else {
        printf("Thread 1: gagal ambil Lock 2. Melepas Lock 1 dan retry\n");
        pthread_mutex_unlock(&lock1);
        retry++;
        usleep(150000);
    }
}

printf("Thread 1 selesai (retry %d kali)\n", retry);
return NULL;
}

void* thread_func2(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&lock2);
        printf("Thread 2: mengambil Lock 2\n");

        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: berhasil mengambil kedua lock!\n");

            sleep(1);

            pthread_mutex_unlock(&lock1);
            pthread_mutex_unlock(&lock2);
            break;
        } else {
            printf("Thread 2: gagal ambil Lock 1. Melepas Lock 2 dan retry\n");
            pthread_mutex_unlock(&lock2);
            retry++;
            usleep(120000);
        }
    }
}

```

```

    printf("Thread 2 selesai (retry %d kali)\n", retry);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== MODIFIKASI DEADLOCK: TRY-LOCK ===\n");

    pthread_create(&t1, NULL, thread_func1, NULL);
    pthread_create(&t2, NULL, thread_func2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai tanpa deadlock\n");
    return 0;
}

```

```

qowi@QOWIAIZANI:~$ ./deadlock_simple
=== MODIFIKASI DEADLOCK: TRY-LOCK ===
Thread 1: mengambil Lock 1
Thread 2: mengambil Lock 2
Thread 1: gagal ambil Lock 2. Melepas Lock 1 dan retry
Thread 2: gagal ambil Lock 1. Melepas Lock 2 dan retry
Thread 2: mengambil Lock 2
Thread 2: berhasil mengambil kedua lock!
Thread 2 selesai (retry 1 kali)
Thread 1: mengambil Lock 1
Thread 1: berhasil mengambil kedua lock!
Thread 1 selesai (retry 1 kali)
Program selesai tanpa deadlock
qowi@QOWIAIZANI:~$

```

3. Bandingkan kedua metode tersebut

1. Konsep Dasar

Lock Ordering

Semua thread *wajib* mengambil resource dalam urutan yang sama, misalnya:

lock1 → lock2

Dengan urutan yang konsisten, **circular wait tidak pernah terjadi**, sehingga deadlock otomatis hilang.

Try-Lock

Thread mencoba mengambil resource kedua menggunakan:

pthread_mutex_trylock()

Jika gagal:

- melepas lock pertama,
- menunggu sebentar,
- mencoba lagi (retry).

Metode ini mencegah *hold and wait*, yaitu sumber penyebab deadlock.

2. Perilaku Program

Lock Ordering

- Tidak terjadi retry.
- Thread mengambil lock sesuai urutan dan langsung berjalan.
- Program berjalan cepat dan stabil.

Try-Lock

- Thread mungkin perlu mengulang berkali-kali.
- Terkadang terjadi saling berebut resource (contention).
- Eksekusi dapat memakan waktu lebih lama.

3. Keunggulan dan Kelemahan

A. Lock Ordering

Keunggulan:

- Paling efisien dan cepat.
- Tidak boros CPU.
- Mudah diprediksi.
- Aman 100% selama urutan konsisten dipakai semua kode.

Kelemahan:

- Memerlukan desain awal yang rapi.
- Semua bagian program harus mengikuti urutan pengambilan resource yang sama.
- Sulit digunakan jika urutan resource tidak dapat ditentukan.

B. Try-Lock

Keunggulan:

- Lebih fleksibel.
- Tidak membutuhkan urutan lock tertentu.
- Cocok untuk kondisi dinamis saat resource tidak selalu sama.

Kelemahan:

- Retry loop bisa memakan waktu.
- Dapat membebani CPU.
- Tidak cocok jika sistem memiliki banyak thread yang berebut resource.

3. Tugas 3 Banker's Algorithm

Buat program baru dengan data:

- 4 proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem.

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
#define P 4 // Jumlah proses
```

```
#define R 3 // Jumlah resource
```

```
// Available resource
```

```
int available[R] = {5, 4, 3};
```

```
// Maximum demand setiap proses
```

```
int maximum[P][R] = {
```

```
    {7, 5, 3},
```

```
    {3, 2, 2},
```



```

    {9, 0, 2},
    {2, 2, 2}
};

// Resource yang sudah dialokasikan
int allocation[P][R] = {
    {0, 1, 0},
    {2, 0, 0},
    {3, 0, 2},
    {2, 1, 1}
};

// Kebutuhan sisa
int need[P][R];

// Hitung need = maximum - allocation
void calculate_need() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = maximum[i][j] - allocation[i][j];
        }
    }
}

// Cek apakah state aman (safe state)
bool is_safe() {
    int work[R];
    bool finish[P] = {false};
    int safe_sequence[P];
    int count = 0;

    // Copy available ke work
    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }

    while (count < P) {
        bool found = false;

        for (int i = 0; i < P; i++) {

```

```

    if (!finish[i]) {
        bool can_allocate = true;

        for (int j = 0; j < R; j++) {
            if (need[i][j] > work[j]) {
                can_allocate = false;
                break;
            }
        }

        // Proses dapat dijalankan
        if (can_allocate) {
            for (int j = 0; j < R; j++) {
                work[j] += allocation[i][j];
            }

            safe_sequence[count++] = i;
            finish[i] = true;
            found = true;
        }
    }

    if (!found) {
        printf("\nState TIDAK aman (unsafe)!\n");
        return false;
    }

    printf("\nState AMAN! Safe Sequence: ");
    for (int i = 0; i < P; i++) {
        printf("P%d ", safe_sequence[i]);
        if (i < P - 1) printf("-> ");
    }
    printf("\n");

    return true;
}

// Request resource

```

```

bool request_resources(int process, int request[]) {
    printf("\nRequest dari P%d: ", process);
    for (int i = 0; i < R; i++) printf("%d ", request[i]);
    printf("\n");

    // Cek request ≤ need
    for (int i = 0; i < R; i++) {
        if (request[i] > need[process][i]) {
            printf("Request > need. Ditolak!\n");
            return false;
        }
    }

    // Cek request ≤ available
    for (int i = 0; i < R; i++) {
        if (request[i] > available[i]) {
            printf("Resource tidak cukup. Ditolak!\n");
            return false;
        }
    }

    // Simulasi alokasi
    for (int i = 0; i < R; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    // Cek apakah masih safe
    if (is_safe()) {
        printf("Request diterima!\n");
        return true;
    } else {
        // Rollback jika unsafe
        printf("Request menyebabkan unsafe state! Rollback...\n");
        for (int i = 0; i < R; i++) {
            available[i] += request[i];
            allocation[process][i] -= request[i];
            need[process][i] += request[i];
        }
    }
}

```

```

        return false;
    }
}

int main() {
    printf("=== BANKER'S ALGORITHM (TUGAS 3) ===\n");

    calculate_need();

    printf("\nCek state awal:\n");
    is_safe();

    // Contoh request aman
    int req1[R] = {1, 0, 2};
    request_resources(1, req1);

    // Contoh request tidak aman
    int req2[R] = {4, 3, 0};
    request_resources(3, req2);

    printf("\nProgram selesai.\n");
    return 0;
}

```

```

qowi@QOWIAIZANI:~$ gcc banker_task3.c -o banker_task3
qowi@QOWIAIZANI:~$ ./banker_task3
=== BANKER'S ALGORITHM (TUGAS 3) ===

Cek state awal:

State AMAN! Safe Sequence: P1 -> P2 -> P3 -> P0

Request dari P1: 1 0 2

State AMAN! Safe Sequence: P1 -> P2 -> P3 -> P0
Request diterima!

Request dari P3: 4 3 0
Request > need. Ditolak!

Program selesai.
qowi@QOWIAIZANI:~$ |

```

4. Tugas 4 Kasus Nyata (Real Case)

Buat simulasi deadlock untuk kasus:

“Dua orang ingin transfer uang antar rekening secara bersamaan”

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

1. PROGRAM DEADLOCK (VERSI DEADLOCK)

File: transfer_deadlock.c

```
#include <stdio.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
```

```
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* transfer_A(void* arg) {
```

```
    printf("A: Mengunci Rekening 1...\n");
```

```
    pthread_mutex_lock(&rekening1);
```

```
    sleep(1);
```

```
    printf("A: Mencoba mengunci Rekening 2...\n");
```

```
    pthread_mutex_lock(&rekening2);
```

```
    printf("A: Transfer selesai!\n");
```

```
    pthread_mutex_unlock(&rekening2);
```

```
    pthread_mutex_unlock(&rekening1);
```

```
    return NULL;
```

```
}
```

```
void* transfer_B(void* arg) {
```

```
    printf("B: Mengunci Rekening 2...\n");
```

```
    pthread_mutex_lock(&rekening2);
```

```
    sleep(1);
```

```
    printf("B: Mencoba mengunci Rekening 1...\n");
```

```
    pthread_mutex_lock(&rekening1);
```

```

    printf("B: Transfer selesai!\n");

    pthread_mutex_unlock(&rekening1);
    pthread_mutex_unlock(&rekening2);
    return NULL;
}

int main() {
    pthread_t A, B;

    printf("=== SIMULASI DEADLOCK TRANSFER UANG ===\n");

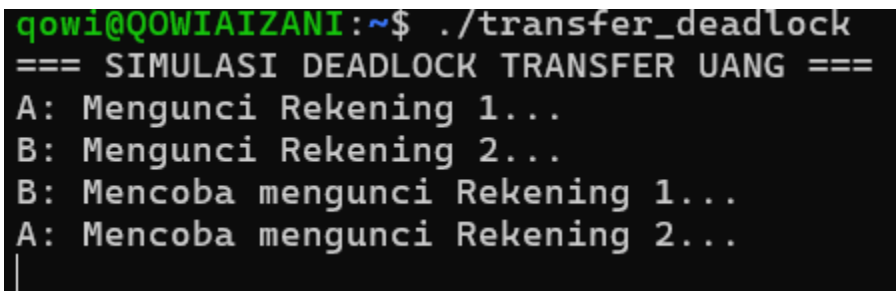
    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}

```

Hasil: Program akan macet (deadlock).



```

qowi@QOWIAIZANI:~$ ./transfer_deadlock
=== SIMULASI DEADLOCK TRANSFER UANG ===
A: Mengunci Rekening 1...
B: Mengunci Rekening 2...
B: Mencoba mengunci Rekening 1...
A: Mencoba mengunci Rekening 2...
|

```

2. PENCEGAHAN DEADLOCK: LOCK ORDERING

Mencegah Circular Wait.

Aturan:

Rekening dengan ID lebih kecil dikunci lebih dulu.

- Rekening 1 selalu dikunci sebelum Rekening 2.

File: transfer_lock_ordering.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

void lock_in_order() {
    pthread_mutex_lock(&rekening1);
    pthread_mutex_lock(&rekening2);
}

void unlock_in_order() {
    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);
}

void* transfer_A(void* arg) {
    printf("A: melakukan transfer...\n");

    lock_in_order();
    printf("A: Transfer berhasil!\n");
    unlock_in_order();

    return NULL;
}

void* transfer_B(void* arg) {
    printf("B: melakukan transfer...\n");

    lock_in_order();
    printf("B: Transfer berhasil!\n");
    unlock_in_order();

    return NULL;
}
```

```

int main() {
    pthread_t A, B;

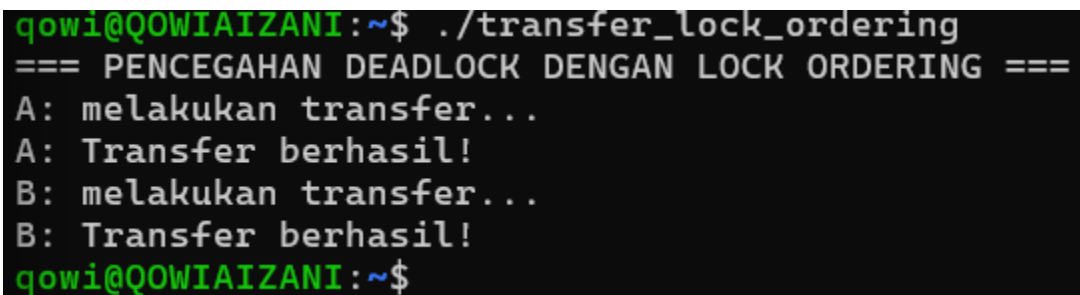
    printf("=== PENCEGAHAN DEADLOCK DENGAN LOCK ORDERING\n");

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}

```



```

qowi@QOWIAIZANI:~$ ./transfer_lock_ordering
=== PENCEGAHAN DEADLOCK DENGAN LOCK ORDERING ===
A: melakukan transfer...
A: Transfer berhasil!
B: melakukan transfer...
B: Transfer berhasil!
qowi@QOWIAIZANI:~$

```

Hasil: Tidak terjadi deadlock, karena kedua thread mengambil lock dengan urutan sama.

3. PENCEGAHAN DEADLOCK: TRY-LOCK + RETRY

Jika gagal ambil lock kedua → lepas lock pertama → coba ulang.

File: transfer_trylock.c

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;

void* transfer_A(void* arg) {

```



```

int retry = 0;

while (1) {
    pthread_mutex_lock(&rekening1);
    printf("A: Mengunci Rekening 1\n");

    if (pthread_mutex_trylock(&rekening2) == 0) {
        printf("A: Transfer berhasil!\n");
        pthread_mutex_unlock(&rekening2);
        pthread_mutex_unlock(&rekening1);
        break;
    }

    printf("A: Gagal mendapatkan Rekening 2, retry...\n");
    pthread_mutex_unlock(&rekening1);
    retry++;
    usleep(100000);
}

printf("A selesai (retry %d)\n", retry);
return NULL;
}

void* transfer_B(void* arg) {
    int retry = 0;

    while (1) {
        pthread_mutex_lock(&rekening2);
        printf("B: Mengunci Rekening 2\n");

        if (pthread_mutex_trylock(&rekening1) == 0) {
            printf("B: Transfer berhasil!\n");
            pthread_mutex_unlock(&rekening1);
            pthread_mutex_unlock(&rekening2);
            break;
        }

        printf("B: Gagal mendapatkan Rekening 1, retry...\n");
        pthread_mutex_unlock(&rekening2);
        retry++;
    }
}

```

```

        usleep(120000);
    }

    printf("B selesai (retry %d)\n", retry);
    return NULL;
}

int main() {
    pthread_t A, B;

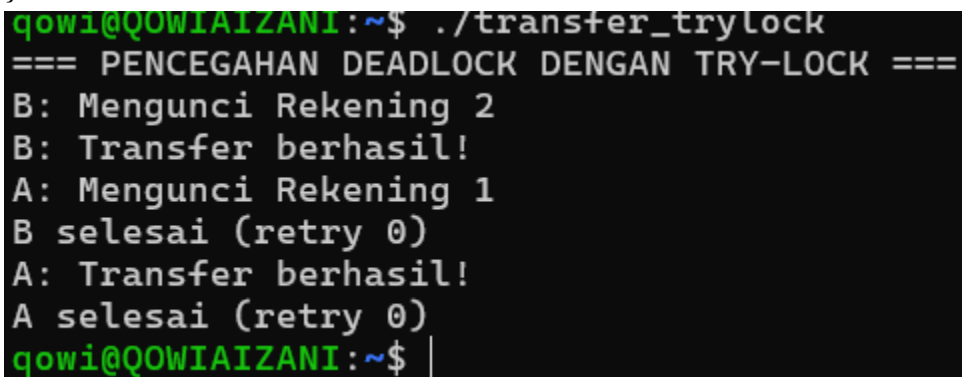
    printf("=== PENCEGAHAN DEADLOCK DENGAN TRY-LOCK ===\n");

    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);

    pthread_join(A, NULL);
    pthread_join(B, NULL);

    return 0;
}

```



```

qowi@QOWIAIZANI:~$ ./transfer_trylock
=== PENCEGAHAN DEADLOCK DENGAN TRY-LOCK ===
B: Mengunci Rekening 2
B: Transfer berhasil!
A: Mengunci Rekening 1
B selesai (retry 0)
A: Transfer berhasil!
A selesai (retry 0)
qowi@QOWIAIZANI:~$ |

```

Hasil: Tidak terjadi deadlock, tetapi retry bisa terjadi beberapa kali.